

**SIMATS ENGINEERING
ASSESSMENT TOOL 3**

COURSE CODE: ITA 06

COURSE NAME: MACHINE LEARNING

COURSE OUTCOME COVERED

CO2: *Neural Network Representation, Perceptron Model, Multilayer Perceptron's, Backpropagation Algorithm, Deep Neural Networks, Genetic Algorithms, Hypothesis Space Search, Genetic Programming, Models of Evaluation and Learning (BL1, BL2)*

Assessment Tool Weightage: 10%

QUESTIONS:15

Total Marks:25

Annexure A

DEBUGGING & OPTIMIZATION TASK QUESTIONS

Code of Conduct:

I Moshitha Kolanjiyappan (192421205) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: Moshitha K

Annexure A

Short Answer Test

1. A perceptron model is trained on a dataset that is known to be linearly separable, yet the algorithm fails to converge even after many iterations. Analyze the possible causes of this issue, including improper learning rate selection, incorrect weight initialization, bias handling errors, or flawed implementation of the update rule. Propose debugging strategies to identify the root cause and suggest optimization techniques to ensure faster and stable convergence.

(5 Marks)

2. While training a multilayer perceptron using the backpropagation algorithm, the network exhibits slow convergence and gets stuck in local minima. Examine the potential reasons such as vanishing gradients, inappropriate activation functions, poor weight initialization, or suboptimal learning rates. Describe step-by-step debugging approaches to trace the issue and recommend optimization techniques like adaptive learning rates, normalization, or advanced optimizers to improve performance.

(5 Marks)

3. A genetic algorithm designed for hypothesis space search is producing inconsistent and suboptimal solutions across multiple runs. Investigate possible issues such as improper encoding of chromosomes, incorrect fitness function design, premature convergence, or lack of diversity in the population. Explain how to debug these problems systematically and suggest optimization strategies like mutation tuning, selection pressure adjustment, and elitism to enhance solution quality and convergence speed.

(5 Marks)

4. A deep neural network combined with genetic programming is used for solving a complex prediction problem, but it suffers from overfitting and high computational cost. Identify debugging steps to analyze issues related to model complexity, over-parameterization, and insufficient training data. Propose optimization techniques such as regularization, dropout, pruning, and efficient evaluation models to balance accuracy and computational efficiency while improving generalization capability.

(5 Marks)

5. A decision tree model trained on a large dataset shows high accuracy on training data but poor performance on unseen data. Analyze possible causes such as overfitting, deep tree structure, or noisy data. Propose debugging steps and optimization methods like pruning, cross-validation, and feature selection to improve generalization.

(5 Marks)

6. A convolutional neural network (CNN) used for image classification produces unstable results across training epochs. Examine potential issues such as improper data augmentation, vanishing gradients, or batch normalization errors. Describe debugging strategies and suggest optimization techniques to stabilize training and improve accuracy.

(5 Marks)

7. A reinforcement learning agent fails to learn an optimal policy in a simulated environment. Investigate causes such as poor reward design, insufficient exploration, or incorrect state representation. Explain debugging methods and recommend improvements like reward shaping, exploration strategies, and tuning hyperparameters.

(5 Marks)

8. A clustering algorithm like K-means produces inconsistent cluster assignments for similar datasets. Analyze possible reasons such as poor initialization, incorrect number of clusters, or sensitivity to outliers. Propose debugging approaches and optimization strategies like K-means++, normalization, and silhouette analysis.

(5 Marks)

9. A support vector machine (SVM) model underperforms on a nonlinear dataset. Identify issues such as incorrect kernel choice, improper parameter tuning, or scaling problems. Suggest debugging techniques and optimization methods including kernel selection, hyperparameter tuning, and feature scaling.

(5 Marks)

10. A natural language processing (NLP) model using recurrent neural networks (RNNs) suffers from poor long-term dependency learning. Examine causes such as vanishing gradients and limited memory capacity. Describe debugging steps and propose solutions like LSTM/GRU units, attention mechanisms, and improved training strategies.

(5 Marks)

11. A support vector machine (SVM) model trained for classification is producing very low accuracy on both training and test datasets. Analyze possible causes such as improper kernel selection, incorrect hyperparameter tuning (C and gamma), or poor feature scaling. Describe systematic debugging steps to identify the issue and suggest optimization techniques like kernel tuning, normalization, and grid search to improve model performance.

(5 Marks)

12. A K-Means clustering algorithm is applied to a dataset, but the clusters formed are highly imbalanced and do not reflect the actual data distribution. Investigate possible issues such as incorrect choice of K, poor initialization of centroids, or presence of outliers. Explain debugging approaches and suggest improvements like the Elbow method, K-Means++, and outlier removal techniques.

(5 Marks)

13. A recurrent neural network (RNN) designed for time-series prediction shows inconsistent outputs and fails to capture long-term dependencies. Analyze possible causes such as vanishing gradients, insufficient sequence length, or improper architecture design. Describe debugging steps and propose optimization techniques such as using LSTM/GRU units, gradient clipping, and sequence normalization.

(5 Marks)

14. A Naïve Bayes classifier used for text classification performs poorly when applied to real-world data containing noisy and irrelevant features. Examine possible issues such as violation of feature independence assumption, improper preprocessing, or imbalanced dataset. Explain debugging strategies and suggest improvements like feature selection, smoothing techniques, and data balancing methods.

(5 Marks)

15. A reinforcement learning agent trained to solve a navigation problem fails to learn an optimal policy and shows erratic behavior. Analyze possible causes such as sparse rewards, improper exploration-exploitation balance, or incorrect reward function design. Describe debugging approaches and recommend optimization strategies like reward shaping, epsilon decay tuning, and policy optimization methods to improve learning performance.

(5 Marks)

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Conceptual Understanding	30	Demonstrates clear and complete understanding of the concept	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor or incorrect understanding
Accuracy of Answer	20	Completely correct answer with no errors	Mostly correct with minor errors	Partially correct with noticeable errors	Incorrect or irrelevant answer
Application of Knowledge	30	Correctly applies concepts to solve the question/problem	Applies concepts with minor mistakes	Limited or partially correct application	No or incorrect application
Completeness of Response	10	Covers all required parts of the question	Covers most parts with minor omissions	Covers only some parts	Incomplete or missing response
Clarity & Presentation	10	Clear, concise, and well-organized answer	Understandable with minor issues	Somewhat unclear or poorly structured	Difficult to understand

1. Perceptron fails to converge

Causes

- Learning rate too high/low.
- Wrong weight initialization.
- Bias term not updated.
- Incorrect perceptron update rule.
- Data not properly normalized.

Debugging

- Verify update equation.
- Print weights after each iteration.
- Check bias updates.
- Plot data to confirm linear separability.
- Monitor classification error.

Optimization

- Use a moderate learning rate (0.01–0.1).
- Initialize small random weights.
- Normalize input features.
- Shuffle training samples each epoch.

Conclusion

Correct learning rate, proper bias handling, normalized data, and a verified update rule ensure faster and stable perceptron convergence.

2. MLP backpropagation slow / stuck in local minima

Causes

- Vanishing gradients.
- Sigmoid/tanh saturation.
- Poor weight initialization.
- Inappropriate learning rate.

Debugging

- Monitor loss curve.
- Check gradient magnitudes.
- Inspect activation outputs.
- Compare training and validation loss.

Optimization

- Use ReLU activation.
- Apply He/Xavier initialization.
- Normalize inputs.
- Use Adam optimizer with adaptive learning rates.

Conclusion

ReLU, proper initialization, normalization, and Adam significantly improve convergence speed and help avoid local minima.

3. Genetic Algorithm gives inconsistent solutions

Causes

- Incorrect chromosome encoding.
- Poor fitness function.
- Premature convergence.

- Low population diversity.

Debugging

- Validate chromosome representation.
- Test fitness values manually.
- Track best and average fitness.
- Measure population diversity.

Optimization

- Increase mutation probability.
- Reduce excessive selection pressure.
- Use elitism.
- Increase population size.

Conclusion

Proper encoding, a meaningful fitness function, controlled mutation, and elitism improve solution quality and convergence consistency.

4. DNN + Genetic Programming overfitting and expensive

Causes

- Too many parameters.
- Very deep network.
- Limited training data.
- Expensive fitness evaluation.

Debugging

- Compare training vs validation accuracy.
- Count model parameters.
- Check learning curves.
- Measure training time per generation.

Optimization

- L2 regularization.
- Dropout.
- Pruning.
- Early stopping.
- Use smaller surrogate evaluation models.

Conclusion

Regularization, dropout, pruning, and efficient evaluation reduce overfitting and computational cost while improving generalization.

5. Decision tree high train accuracy, low test accuracy

Causes

- Overfitting.
- Excessive tree depth.
- Noisy or irrelevant features.

Debugging

- Compare train/test accuracy.
- Visualize tree depth.
- Inspect feature importance.
- Identify noisy attributes.

Optimization

- Pre-pruning (max_depth, min_samples_split).
- Post-pruning.
- Cross-validation.
- Feature selection.

Conclusion

Pruning and cross-validation reduce overfitting and improve decision-tree generalization.

6. CNN unstable across epochs

Causes

- Aggressive data augmentation.
- Vanishing/exploding gradients.
- Incorrect batch normalization settings.
- Learning rate too high.

Debugging

- Check sample augmented images.
- Monitor gradient norms.
- Track batch-normalization statistics.
- Observe training/validation loss.

Optimization

- Use moderate augmentation.
- Add Batch Normalization.
- Use ReLU activation.
- Apply learning-rate scheduling and gradient clipping.

Conclusion

Batch normalization, proper augmentation, and controlled learning rates stabilize CNN training and improve accuracy.

7. Reinforcement learning agent fails to learn

Causes

- Poor reward design.
- Insufficient exploration.
- Incorrect state representation.
- Weak hyperparameter settings.

Debugging

- Visualize reward over episodes.
- Inspect visited states.
- Test random and greedy policies.
- Check Q-values or policy outputs.

Optimization

- Reward shaping.
- ϵ -greedy or entropy-based exploration.
- Better state features.
- Tune learning rate and discount factor.

Conclusion

Good reward design, sufficient exploration, and tuned hyperparameters help the agent learn an optimal policy.

8. K-means inconsistent clusters

Causes

- Random centroid initialization.
- Wrong number of clusters.
- Sensitivity to outliers.

- Different feature scales.

Debugging

- Run multiple initializations.
- Plot clusters.
- Check feature ranges.
- Identify outliers.

Optimization

- K-means++ initialization.
- Feature normalization.
- Elbow method.
- Silhouette analysis.
- Remove or cap outliers.

Conclusion

K-means++ with normalized data and an appropriate K produces more stable and meaningful clusters.

9. SVM underperforms on nonlinear data

Causes

- Linear kernel on nonlinear data.
- Poor C and gamma values.
- Unscaled features.

Debugging

- Visualize class distribution.
- Compare kernels.
- Check feature ranges.
- Evaluate validation accuracy.

Optimization

- Use RBF or polynomial kernel.
- Standardize features.
- Tune C and gamma using grid search.

Conclusion

Proper kernel selection, feature scaling, and hyperparameter tuning greatly improve SVM performance on nonlinear datasets.

10. RNN poor long-term dependency learning

Causes

- Vanishing gradients.
- Limited memory.
- Very long sequences.

Debugging

- Monitor gradient norms.
- Test different sequence lengths.
- Evaluate performance on long-range patterns.

Optimization

- Replace RNN with LSTM or GRU.
- Use gradient clipping.
- Normalize sequences.
- Add attention mechanisms.

Conclusion

LSTM/GRU, gradient clipping, and attention enable effective learning of long-term dependencies.

11. SVM low accuracy on both train and test sets

Causes

- Wrong kernel.
- Poor C/γ tuning.
- Unscaled features.
- Weak feature representation.

Debugging

- Check feature scaling.
- Compare kernels.
- Perform cross-validation.
- Inspect confusion matrix.

Optimization

- Standardize data.
- Use RBF kernel.
- Apply GridSearchCV for C and γ .

- Select informative features.

Conclusion

Scaling plus systematic kernel and hyperparameter tuning improves both training and testing accuracy.

12. K-means imbalanced clusters

Causes

- Incorrect K.
- Poor centroid initialization.
- Outliers skewing centroids.
- Unequal cluster densities.

Debugging

- Plot cluster sizes.
- Visualize centroids.
- Detect outliers.
- Evaluate silhouette score.

Optimization

- Elbow method for K.
- K-means++.
- Remove outliers.
- Normalize features.

Conclusion

Choosing the correct K, using K-means++, and handling outliers produce balanced and representative clusters.

13. RNN inconsistent time-series outputs

Causes

- Vanishing gradients.
- Short sequence windows.
- Inadequate architecture.

Debugging

- Plot predictions vs actual values.
- Monitor gradients.

- Test different window sizes.
- Compare training and validation error.

Optimization

- Use LSTM/GRU.
- Apply gradient clipping.
- Normalize sequences.
- Increase sequence length appropriately.

Conclusion

LSTM/GRU with normalized sequences and gradient clipping provides more stable and accurate time-series predictions.

14. Naïve Bayes poor text classification

Causes

- Irrelevant/noisy words.
- Violation of feature independence.
- Imbalanced classes.
- Inadequate preprocessing.

Debugging

- Inspect frequent tokens.
- Check class distribution.
- Evaluate confusion matrix.
- Compare performance with/without stop-word removal.

Optimization

- Stop-word removal and stemming/lemmatization.
- TF-IDF features.
- Laplace smoothing.
- Class balancing (oversampling/undersampling).

Conclusion

Good preprocessing, TF-IDF, smoothing, and class balancing substantially improve Naïve Bayes text classification.

15. RL navigation agent shows erratic behavior

Causes

- Sparse rewards.
- Poor exploration–exploitation balance.
- Incorrect reward function.
- Unstable policy updates.

Debugging

- Plot episode rewards.
- Visualize agent trajectories.
- Inspect action probabilities.
- Test deterministic vs exploratory policies.

Optimization

- Reward shaping.
- Tune ϵ -decay schedule.
- Increase exploration initially.
- Use policy optimization methods (PPO/A2C).
- Adjust learning rate and discount factor.

Conclusion

Reward shaping, well-tuned exploration, and stable policy optimization methods improve learning speed and produce a reliable navigation policy.